Лекція №19 (2 години). Управління глобальним станом (Context API / Zustand).

План лекції

1. Проблема передачі пропсів: що таке Prop Drilling?
2. Локальний стан проти Глобального стану.
3. Вбудоване рішення: React Context API.
4. Створення Контексту та Провайдера.
5. Споживання даних за допомогою хука `useContext`.
6. Недоліки Context API для складних додатків.
7. Знайомство з сучасним стейт-менеджером Zustand (створення Store).

Перелік умовних скорочень

- **State** — стан (пам'ять) додатка.
- **Props** — властивості, що їх батьківський компонент передає дочірньому.
- **API** — інтерфейс прикладного програмування.
- **Store** — централізоване сховище даних.

Вступ

Ми знаємо, що стан у React прив'язаний до компонента, де викликаний хук `useState`. Якщо нам потрібно передати цей стан іншим компонентам, ми спускаємо його "вниз по дереву" через Пропси (Props) від Батька до Дітей.

Але що робити, якщо нашому додатку потрібні дані про поточного користувача (його аватарку чи статус авторизації) на **абсолютно кожній сторінці**? Аватарка має показуватися в `<Header>`, у вікні `<ProfileSettings>`, і біля кожного залишеного коментаря у `<CommentBlock>`. Передавати об'єкт `user` через Props через десятки посередників, яким цей об'єкт насправді не потрібен — це жахливий архітектурний біль.

Для вирішення цієї проблеми були розроблені інструменти управління **Глобальним станом** (State Managers). Мета цієї лекції — розібрати вбудований інструмент React Context API, який підходить для простих речей (як темна/світла тема), та познайомитися із сучасною, легкою альтернативою — бібліотекою **Zustand**.

1. Проблема Prop Drilling (Свердління пропсів)

Уявіть, що у нас є компонент `App` (Батько). Всередині нього є `Layout`, всередині нього — `MainContent`, потім `Article`, потім `ArticleFooter`, і аж всередині `ArticleFooter` знаходиться компонент `AuthorAvatar`, якому потрібне посилання на картинку користувача `user.avatar`.

```
// Щоб передати дані в AuthorAvatar, нам доведеться прокинути їх через ВСІХ
посередників
<App>
  <Layout user={user}>
    <MainContent user={user}>
      <Article user={user}>
        <ArticleFooter user={user}>
          <AuthorAvatar src={user.avatar} />{" "}
          {/* ТІЛЬКИ ТУТ він реально потрібен! */}
        </ArticleFooter>
      </Article>
    </MainContent>
  </Layout>
</App>
```

Ця проблема, коли ви змушені передавати Пропси глибоко по дереву через компоненти, яким ці пропси не потрібні (вони працюють просто як кур'єри), називається **Prop Drilling**. Це засмічує код, ускладнює його читання і робить рефакторинг неможливим.

2. Локальний стан проти Глобального

- **Локальний стан (`useState`):** Це дані, які потрібні ТІЛЬКИ одному компоненту або його найближчим сусідам. Наприклад: відкрите чи закрите модальне вікно, поточний текст в інпуті форми Пошуку. Не треба тягнути їх глобально!
- **Глобальний стан:** Дані, доступу до яких потребують багато не пов'язаних між собою компонентів у різних частинах додатка екрану. Наприклад:
  - Дані про авторизацію (`user`, `token`).
  - Кошик інтернет-магазину (щоб лічильник товарів був у Хедері, а самі товари на сторінці Оформлення).
  - Тема UI (Світла/Темна).
  - Обрана мова інтерфейсу (ua/en).

3. React Context API: Вбудоване рішення

Щоб вирішити проблему Prop Drilling без використання сторонніх бібліотек, React пропонує власний інструмент — **Context API**. Він працює як своєрідний "тунель" або "портал": ви кладете дані в обгортку на самій вершині `App` (Провайдер), а будь-який компонент на будь-якій глибині може одягнути "навушники" (Споживач) і миттєво почути ці дані.

Крок 1. Створення Контексту

Спочатку ми створюємо файл для нашого контексту (наприклад `ThemeContext.jsx`). За допомогою вбудованої функції `createContext`.

```
import { createContext } from "react";

// Створюємо порожню коробку-контекст.
// Можна вказати дефолтне значення (напр. 'light'), що буде використано, якщо не
буде Провайдера
export const ThemeContext = createContext("light");
```

Крок 2. Провайдер (Provider)

Щоб наповнити цю коробку реальними даними, які будуть змінюватись (`useState`), ми робимо компонент обгортку:

```
// файл ThemeContext.jsx
import { createContext, useState } from "react";

export const ThemeContext = createContext(); // Експортуємо саму "коробку"

// Це наш кастомний компонент-обгортка
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
    setTheme(theme === "light" ? "dark" : "light");
  };

  // Все, що ми покладемо в атрибут `value`, стане доступним глобально
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
```

```
      {/* Всі дитячі компоненти додатка */}
      {children}
    </ThemeContext.Provider>
  );
}
```

В найвищому файлі `main.jsx` ми огортаємо весь додаток цією коробкою:

```
<ThemeProvider>
  <App />
</ThemeProvider>
```

Крок 3. Споживання за допомогою хука `useContext`

Тепер кнопка, що лежить десь дуже глибоко в `Header`, може спожити цей контекст і отримати дані напряму без Props!

```
import { useContext } from "react";
import { ThemeContext } from "./ThemeContext"; // Імпортуємо ту саму "коробку"

function ThemeSwitcherButton() {
  // Магія: отримуємо змінні з "порталу"
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <button onClick={toggleTheme}>
      {theme === "light" ? "🌚 Зробити Темним" : "🌞 Зробити Світлим"}
    </button>
  );
}
```

4. Недоліки Context API

Для простих речей (Тема, Мова, Користувач) Context підходить ідеально. Але для великих обсягів даних, що часто змінюються (як список постійно оновлюваних котирувань акцій чи великий складний кошик), він має серйозну архітектурну проблему — **Проблему зайвих рендерів**.

Якщо ви зміните хоча б одну маленьку властивість з 50-ти, що лежать у значенні (value) Provider-а, **УСІ компоненти**, які споживають цей контекст, **перемалюються повністю**, навіть якщо їм конкретно ця

маленька властивість не була потрібна. Додаток може почати гальмувати.

Раніше для складних задач абсолютно всі використовували **Redux**. Але Redux мав настільки "страшний" і багатослівний код налаштування (Boilerplate), що всі його ненавиділи.

Тому сьогодні індустрія перейшла до легких і потужних Стейт-менеджерів, і найпопулярнішим серед них зараз є **Zustand** ("стан" німецькою).

5. Знайомство зі стейт-менеджером Zustand

Zustand — це крихітна стороння бібліотека, яка вирішує всі проблеми Контексту. Її головні плюси:

- Не потрібні компоненти-обгортки Провайдери!
- Оновлюються лише ті компоненти, які "слухають" конкретну змінну, а не все підряд.
- Код створюється за 5 рядків без жодного зайвого шаблону.

Встановлення:

```
npm install zustand
```

Крок 1. Створення Глобального Store

Замість "провайдера і контексту" ми створюємо один центральний `Store` (склад) за допомогою функції `create`. В ньому будуть жити як дані (state), так і функції для їхньої зміни (actions).

```
// файл store/cartStore.js
import { create } from "zustand";

export const useCartStore = create((set) => ({
  // 1. Початковий стан
  cartCount: 0,
  items: [],

  // 2. Функції зміни стану (Actions)
  // Функція set оновлює конкретне поле і зливає його зі старим станом
  addToCart: (productName) => {
    set((state) => ({
      cartCount: state.cartCount + 1,
      items: [...state.items, productName],
    }));
  },
```

```
  clearCart: () => set({ cartCount: 0, items: [] }),
}));
```

Крок 2. Використання Store у будь-якому компоненті

Нашому компоненту не потрібен доступ до всього Сховища. Він отримує свій власний хук, і витягує з нього ЛИШЕ те, що треба.

```
// Компонент Хедера (потрібен тільки лічильник)
import { useCartStore } from "../store/cartStore";

function HeaderBlock() {
  // Компонент Хедера підписується ТІЛЬКИ на `cartCount`.
  // Якщо зміняться `items`, він не буде перемальовуватися! Ось в чому сила
Zustand.
  const cartCount = useCartStore((state) => state.cartCount);

  return <div>Кошик: {cartCount} товарів</div>;
}

// Компонент Купівлі (потрібна функція додавання)
function BuyButton({ product }) {
  // Цей компонент дістав тільки функцію (Action)
  const addToCart = useCartStore((state) => state.addToCart);

  return <button onClick={() => addToCart(product.name)}>Купити</button>;
}
```

Все! Ніяких Провайдерів у `App.jsx`, ніяких зайвих перемальовок. Zustand робить глобальний стан чистим і неймовірно швидким.

Висновки

1. **Locals State (`useState`)** слід використовувати для локальних даних UI-елементів. Концепція **Global State** необхідна лише тоді, коли однаковими даними паралельно користуються далекі компоненти, щоб уникнути проблеми "свердління пропсів" (**Prop Drilling**).
2. React містить вбудований API для базового глобального стану — **Context API**.
3. Для роботи Контексту потрібно огорнути дерево компонентів через `<MyContext.Provider value={...}>`, і використовувати інструмент-приймач `useContext(MyContext)` будь-де нижче в ієрархії.

4. Головний недолік Context API — він може створювати зайві рендери всіх підключених підписників при зміні будь-якого фрагмента своїх глобальних значень.
5. Сучасний підхід до вирішення цієї проблеми лежить через використання легковагових атомарних стейт-менеджерів, таких як **Zustand**.
6. **Zustand** дозволяє позбутися Провайдерів в обгортках, написати Store за 5 рядків коду, гарантувати відсутність зайвих перемальовок і зберігати функції зміни стану разом із самими даними в єдиному місці.

Джерела

1. React Docs: Passing Data Deeply with Context — як і навіщо придумали Context API.
2. Zustand Official Documentation — офіційний сайт і туторіал найсучаснішого інструменту у React-екосистемі.

Запитання для самоперевірки

1. Змоделюйте і опишіть ситуацію, яка ілюструє жах архітектурного паттерну під назвою `Prop Drilling`.
2. Які дані має сенс залишати локальними (через звичайний хук `useState`), а які доцільно винести на Глобальний рівень?
3. Які два ключові кроки (компоненти/хуки) потрібно реалізувати розробнику, щоб створити свій перший Context API та доставити в нього інформацію?
4. Назвіть функції компонента `<ThemeContext.Provider>`, навіщо йому атрибут `value`?
5. У чому полягає критична слабкість Context API під час керування великим об'єктом стану, який містить 50 різних полів і щомиті оновлюється?
6. Назвіть 2 головні відмінності і переваги бібліотеки `Zustand` над класичним рішенням з Context API, коли мова заходить про швидке розгортання проєкту.